

BERT NLTK

```
import nltk
import random
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from transformers import BertTokenizer, BertForSequenceClassification,
Trainer, TrainingArguments, DataCollatorWithPadding
from datasets import Dataset
from sklearn.metrics import accuracy_score,
precision_recall_fscore_support

W0702 12:45:21.989000 28820 site-packages\torch\distributed\elastic\
multiprocessing\redirects.py:29] NOTE: Redirects are currently not
supported in Windows or MacOS.

# Download NLTK movie_reviews dataset
nltk.download('movie_reviews')

from nltk.corpus import movie_reviews

# Prepare the dataset
documents = [(movie_reviews.raw(fileid), category)
              for category in movie_reviews.categories()
              for fileid in movie_reviews.fileids(category)]

random.shuffle(documents)

# Convert to DataFrame
df = pd.DataFrame(documents, columns=['text', 'tag'])

# Encode labels
label_encoder = LabelEncoder()
df['label'] = label_encoder.fit_transform(df['tag']) # pos → 1, neg → 0

# Train-test split
train_texts, test_texts, train_labels, test_labels = train_test_split(
    df['text'].tolist(), df['label'].tolist(), test_size=0.2,
    random_state=42
)

[nltk_data] Downloading package movie_reviews to
[nltk_data] C:\Users\Madhu\AppData\Roaming\nltk_data...
[nltk_data] Package movie_reviews is already up-to-date!

# Load tokenizer
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
```

```

# Tokenize data
train_encodings = tokenizer(train_texts, truncation=True,
padding=True)
test_encodings = tokenizer(test_texts, truncation=True, padding=True)

# Convert to HuggingFace Datasets
train_dataset = Dataset.from_dict({
    'input_ids': train_encodings['input_ids'],
    'attention_mask': train_encodings['attention_mask'],
    'label': train_labels
})
test_dataset = Dataset.from_dict({
    'input_ids': test_encodings['input_ids'],
    'attention_mask': test_encodings['attention_mask'],
    'label': test_labels
})

# Load model
model = BertForSequenceClassification.from_pretrained("bert-base-
uncased", num_labels=2)

# Compute metrics function
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = logits.argmax(axis=-1)
    precision, recall, f1, _ = precision_recall_fscore_support(labels,
predictions, average='binary')
    acc = accuracy_score(labels, predictions)
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

# Training arguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=1,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    evaluation_strategy="epoch",
    save_strategy="no",
    logging_dir='./logs',
    logging_steps=10
)

# Trainer setup
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

```

```
trainer = Trainer(  
    model=model,  
    args=training_args,  
    train_dataset=train_dataset,  
    eval_dataset=test_dataset,  
    tokenizer=tokenizer,  
    data_collator=data_collator,  
    compute_metrics=compute_metrics  
)
```

Train

```
trainer.train()
```

Evaluate

```
metrics = trainer.evaluate()  
print(metrics)
```

```
C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\huggingface_hub\  
file_download.py:943: FutureWarning: `resume_download` is deprecated  
and will be removed in version 1.0.0. Downloads always resume when  
possible. If you want to force a new download, use  
`force_download=True`.
```

```
warnings.warn(  
Some weights of BertForSequenceClassification were not initialized  
from the model checkpoint at bert-base-uncased and are newly  
initialized: ['classifier.bias', 'classifier.weight']  
You should probably TRAIN this model on a down-stream task to be able  
to use it for predictions and inference.
```

```
C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\torch\utils\data\  
dataloader.py:665: UserWarning: 'pin_memory' argument is set as true  
but no accelerator is found, then device pinned memory won't be used.  
warnings.warn(warn_msg)
```

<IPython.core.display.HTML object>

```
C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\torch\utils\data\  
dataloader.py:665: UserWarning: 'pin_memory' argument is set as true  
but no accelerator is found, then device pinned memory won't be used.  
warnings.warn(warn_msg)
```

<IPython.core.display.HTML object>

```
{'eval_loss': 0.3873242139816284, 'eval_accuracy': 0.845,  
'eval_precision': 0.8232558139534883, 'eval_recall':  
0.8805970149253731, 'eval_f1': 0.8509615384615384, 'eval_runtime':  
193.8017, 'eval_samples_per_second': 2.064, 'eval_steps_per_second':  
0.067, 'epoch': 1.0}
```

BERT LOCAL DATASET

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score,
precision_recall_fscore_support
from transformers import
(BertTokenizer, BertForSequenceClassification, Trainer, TrainingArguments
, DataCollatorWithPadding)
from datasets import Dataset
import numpy as np

# === 1. Load Dataset === #
df = pd.read_csv('movie_review.csv')
df = df.sample(2000) # optional sampling for speed
label_encoder = LabelEncoder()
df['label'] = label_encoder.fit_transform(df['tag'])

# === 2. Train-Test Split === #
train_texts, test_texts, train_labels, test_labels = train_test_split(
    df['text'].tolist(), df['label'].tolist(), test_size=0.2,
    random_state=42)

# === 3. Tokenization === #
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
train_encodings = tokenizer(train_texts, truncation=True,
padding=True)
test_encodings = tokenizer(test_texts, truncation=True, padding=True)

# === 4. Create HF Datasets === #
train_dataset = Dataset.from_dict({
    'input_ids': train_encodings['input_ids'],
    'attention_mask': train_encodings['attention_mask'],
    'label': train_labels
})
test_dataset = Dataset.from_dict({
    'input_ids': test_encodings['input_ids'],
    'attention_mask': test_encodings['attention_mask'],
    'label': test_labels
})

# === 5. Load Model === #
model = BertForSequenceClassification.from_pretrained("bert-base-
uncased", num_labels=2)

# === 6. Define Evaluation Metrics === #
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
```

```

    acc = accuracy_score(labels, preds)
    precision, recall, f1, _ = precision_recall_fscore_support(labels,
preds, average='binary')
    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

```

=== 7. Training Arguments ===

```

training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=2,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=32,
    evaluation_strategy="epoch",
    save_strategy="no",
    logging_dir='./logs',
    logging_steps=10
)

```

=== 8. Trainer Setup ===

```

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)

```

=== 9. Training ===

```

trainer.train()

```

=== 10. Evaluation ===

```

metrics = trainer.evaluate()
print("Evaluation Metrics:")
for k, v in metrics.items():
    print(f"{k}: {v:.4f}")

```

C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\huggingface_hub\file_download.py:943: FutureWarning: `resume\_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. If you want to force a new download, use `force\_download=True`.

```

warnings.warn(
Some weights of BertForSequenceClassification were not initialized

```

from the model checkpoint at bert-base-uncased and are newly initialized: ['classifier.bias', 'classifier.weight']
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.

C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\torch\utils\data\loader.py:665: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
warnings.warn(warn_msg)

<IPython.core.display.HTML object>

C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\torch\utils\data\loader.py:665: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
warnings.warn(warn_msg)

C:\Users\Madhu\anaconda3\envs\FDS\Lib\site-packages\torch\utils\data\loader.py:665: UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
warnings.warn(warn_msg)

<IPython.core.display.HTML object>

Evaluation Metrics:
eval_loss: 0.6458
eval_accuracy: 0.6450
eval_precision: 0.6959
eval_recall: 0.5694
eval_f1: 0.6263
eval_runtime: 165.3087
eval_samples_per_second: 2.4200
eval_steps_per_second: 0.0790
epoch: 2.0000